

Elasticity-Only Modeling for Randomized Motor-Insurance Price Tests

Research report, real-data LLM agent playbook, and Python reference implementation

OpenAI
5 August 2026

***Evidence boundary.** This document contains no synthetic insurance-model leaderboard and no empirical claim about the user's portfolio. All model selection instructions require real randomized quote-level data, customer-disjoint out-of-fold predictions, and an untouched chronological holdout.*

Contents

- **Part I — Research report:** estimand, model hierarchy, validation, Optuna design, and recommendation.
- **Part II — LLM agent operating instruction:** an imperative end-to-end procedure for real insurance data.
- **Part III — Real-data contract:** mandatory schema, feature timing, and stop conditions.
- **Part IV — Research log:** positive findings, qualifications, dead ends, and unresolved questions.
- **Part V — Software QA:** test scope, environment, limitations, and evidence boundary.
- **References:** case-only methodology, heterogeneous-effect evaluation, policy evaluation, and official library documentation.

Part I — Research report

Executive conclusion

The business objective is not primarily to predict whether a customer buys. It is to identify quote contexts in which a tested price increase or decrease produces the best relative contribution. That objective can be addressed without a customer-level base-conversion model.

For arm t , let $q_t(x)$ be conversion and let $g_t(x)$ be contribution if purchased. Relative to control,

$$q_t(x) = q_C(x) \text{ RR}_t(x).$$

The expected contribution is therefore

$$g_t(x) q_t(x) = q_C(x) g_t(x) \text{ RR}_t(x).$$

The common factor $q_C(x)$ cancels from the price choice. Among the tested arms, the optimal quote-level action is

$$\text{argmax}_t g_t(x) \text{ RR}_t(x).$$

This supports an elasticity-only research program. The randomized experiment still needs all eligible quotes for auditing and policy evaluation, but the primary shape estimator can be trained on buyers through the exact Bayes-flip conditional likelihood.

The most promising practical strategy is a **model ladder**, not a single assumed winner:

1. global scalar elasticity as the mandatory anchor;
2. penalized pairwise logistic as an interpretable heterogeneous baseline;
3. calibrated H/C and L/C LightGBM models with propensity offsets;
4. calibrated H/C and L/C XGBoost models with propensity offsets;
5. heavily smoothed pairwise Random Forest as a robustness benchmark;
6. scalar custom-objective LightGBM/XGBoost and Random-Forest moment models;
7. a constrained convex ensemble of real out-of-fold predictions.

The winner must be selected on real nested out-of-fold and chronological-holdout evidence. No synthetic model ranking is used or supplied.

1. Target, identification, and interpretation

1.1 Bayes-flip identity

Let the randomized arm be T in $\{H, C, L\}$ and purchase be Y . The row-level randomization probabilities are $\pi_t(x)$. Define

$$\begin{aligned} q_t(x) &= P(Y=1 \mid T=t, X=x) \\ r_t(x) &= P(T=t \mid Y=1, X=x). \end{aligned}$$

Bayes' rule gives

$$r_t(x) = \pi_t(x) q_t(x) / \sum_j \pi_j(x) q_j(x).$$

Consequently,

$$\begin{aligned} \text{RR}_t(x) &= q_t(x) / q_C(x) \\ &= [r_t(x) / \pi_t(x)] / [r_C(x) / \pi_C(x)]. \end{aligned}$$

This is exact and does not rely on conversion being rare. The randomized-trial case-only literature gives a related relative-risk interpretation of treatment assignment among cases [1].

For fixed 10/80/10 assignment,

$$\begin{aligned} \text{RR}_{HC} &= 8r_H / r_C \\ \text{RR}_{LC} &= 8r_L / r_C. \end{aligned}$$

The economically natural monotonicity condition is

$$RR_HC \leq 1 \leq RR_LC,$$

which is equivalent to

$$r_H/\pi_H \leq r_C/\pi_C \leq r_L/\pi_L.$$

1.2 What is and is not identified

The buyer-side posterior identifies conditional relative conversion risks. It does not identify absolute $q_C(x)$ by itself. For the present business objective, that is not fatal because $q_C(x)$ cancels from the arm choice.

The learned function should be described as a **conditional marginal finite-difference response**. If unobserved tariff competitiveness affects both baseline conversion and price sensitivity, the estimated response averages over that unobserved state. It is not the latent willingness-to-pay of a physically identifiable individual.

Mandatory insurance does not imply mandatory purchase from this insurer. The outcome is own-quote acceptance, including substitution to competitors, abandonment, delay, and channel switching.

1.3 Finite-difference elasticity

For nominal multipliers 1.10, 1.00, and 0.90,

$$\begin{aligned} \text{beta_H}(x) &= -\log RR_HC(x)/\log(1.10) \\ \text{beta_L}(x) &= -\log RR_LC(x)/\log(0.90). \end{aligned}$$

These are adjacent finite-difference magnitudes. A scalar model assumes

$$\log RR_tC(x) = -\text{beta}(x)[\log P_t - \log P_C], \text{beta}(x) \geq 0.$$

This is a useful regularizer for a three-price experiment, but it must be tested against separate H/C and L/C responses.

2. Why buyer-only learning is attractive and where it can fail

2.1 Advantage

A full binary conversion loss is dominated by baseline purchase prediction when conversion is around 5%. The buyer conditional likelihood removes the common conversion scale and places all optimization pressure on the randomized arm composition among purchasers. It also makes the known treatment propensity an offset rather than a relationship the learner must rediscover.

This is a robustness advantage under nuisance misspecification. It is not a claim that conditioning creates information. A correctly specified full likelihood remains at least as informative as its conditional component.

2.2 Buyer-distribution weighting

The conditional likelihood weights regions of feature space according to where buyers occur. This is often aligned with contribution: errors in regions with almost no sales matter less economically. If the scientific goal instead requires an elasticity surface representative of all quote opportunities, one could reweight buyers toward the quote distribution, but that requires a conversion or density-ratio model and can cause high variance. That is an optional sensitivity analysis, not the default.

2.3 Major failure modes

Buyer-only learning can fail when:

- propensity logging is wrong;
- post-randomization variables leak arm identity;
- one customer appears across folds;
- sample sizes in side-arm buyers are too small for heterogeneity;
- price response changes with unmeasured competitive position;
- the model is selected by ranking metrics without probability calibration;
- class balancing changes the buyer-arm posterior;

- tree models fit random arm imbalances in high-dimensional feature space.

These risks justify strong pooling, calibration, forward validation, and explicit model-selection gates.

3. Recommended model hierarchy

3.1 Global models

Fit three global references:

1. no effect: $RR_{HC}=RR_{LC}=1$;
2. unrestricted global two-gap model;
3. global scalar $\beta \geq 0$.

The flexible model must improve over the global model out of fold. A statistically detectable average effect does not imply learnable heterogeneity.

3.2 Pairwise conditional models

For t equal to H or L, among buyers in $\{t, C\}$:

$$\begin{aligned} \text{logit } P(T=t \mid T \in \{t, C\}, Y=1, X) \\ = \log(\pi_t/\pi_C) + \log RR_{tC}(X). \end{aligned}$$

This suggests two binary models. They are coherent after reconstructing

$$r = \text{softmax}(\log \pi + [\log RR_{HC}, 0, \log RR_{LC}]).$$

Pairwise models have practical advantages:

- they use standard binary learners;
- each contrast can have different regularization;
- calibration is easy to inspect;
- upper and lower effects can differ;
- equal-contrast scoring prevents the control arm from hiding side-arm errors.

3.3 LightGBM and XGBoost with offsets

For XGBoost, use the propensity log odds as `base_margin`; for LightGBM use `init_score`. Both learners then estimate residual log risk ratio. XGBoost documents custom objectives as requiring row-additive, smooth losses and discusses Hessian constraints and Fisher approximations [12–14]. LightGBM exposes `init_score` in its Dataset API and emphasizes the interaction of leaves, depth, and minimum leaf size in tuning [10–11].

For pairwise models, native binary logistic loss plus an offset is preferable to a custom objective. It is simpler, exact, and uses library-tested derivatives.

3.4 Random Forest

Random Forest has no fixed logit offset in the scikit-learn API. Inverse-propensity weighting among pairwise buyers yields a target whose logit is the risk ratio. The forest should use large leaves, shallow-to-moderate depth, `class_weight=None`, and out-of-fold calibration. scikit-learn allows `log_loss`, `min_samples_leaf`, feature subsampling, bootstrap subsampling, and cost-complexity pruning [8].

Random Forest is unlikely to be the most statistically efficient model with few side-arm buyers, but it is useful because its inductive bias differs from boosting. Agreement between a smoothed forest and boosted models is stronger evidence than a single tree family winning by a tiny score margin.

3.5 Scalar custom objective

The scalar buyer likelihood is convex in β for fixed features. With $\beta = \text{softplus}(f)$, use the gradient

```
[z_T-E_r(z)] sigmoid(f)
```

and positive Fisher curvature

```
Var_r(z) sigmoid(f)^2+epsilon.
```

The raw-score exact Hessian can be negative because of the positivity transform, so the Fisher Hessian is the stable tree-boosting choice. Keep trees very shallow and use strong shrinkage.

3.6 Random-Forest moment inversion

Inverse-propensity-weighted buyer arm dose has a conditional mean that is monotone in scalar β . A RandomForestRegressor can estimate that mean and invert it by bisection. This gives Random Forest a one-dimensional elasticity target without inventing an individual label. It is less efficient than maximum likelihood but valuable as a structurally different benchmark.

3.7 Multiclass and ensemble models

An inverse-propensity multiclass classifier estimates normalized arm risks. It is a benchmark rather than the first choice because it does not directly exploit the two contrasts or monotonicity.

After OOF predictions exist, fit a convex stack in log-risk-ratio space. Include the global scalar anchor. Recalibrate the stack and enforce signs. This frequently gives a useful variance-control mechanism: when flexible models disagree, the global model receives more weight.

4. Calibration is part of the model

Elasticity is a function of odds ratios, so probability miscalibration is amplified. scikit-learn's calibration guide recommends calibration on independent data and warns that isotonic calibration overfits when the calibration sample is small; its documentation also notes characteristic calibration behavior of Random Forest [6-7].

Fit an affine risk-ratio calibrator on inner OOF predictions:

```
log RR_cal=alpha+s(raw-center), s>=0.
```

This simultaneously calibrates the average effect and shrinks heterogeneous dispersion. A slope near zero is a principled result: the base model has not learned reproducible heterogeneity.

Use 4-5 large grouped calibration bins. Do not rely only on calibration slope, because a globally linear correction can hide local failures.

Monotonicity should be enforced after calibration. Report raw violation rate and projection distance. Post-processing is a safety layer, not evidence that the unconstrained model was correct.

5. Evaluation focused on elasticity

5.1 Proper likelihood scores

The primary scores are:

- buyer multiclass NLL;
- H/C pairwise conditional NLL;
- L/C pairwise conditional NLL;
- equal-contrast NLL, the average of the two pair losses.

Cross-entropy is a proper score for the probabilities from which risk ratios are derived. VUS, multiclass AUC, and accuracy measure discrimination but not numerical probability ratios. They can be secondary diagnostics, never selection criteria.

5.2 Information gain over global response

For each outer fold, fit the global response on the outer training set and compare held-out NLL. Report the gain in millinats per pair buyer with a customer-clustered paired interval. This isolates the question that matters: do features locate meaningful elasticity differences beyond the portfolio average?

5.3 Calibration and heterogeneity

Use held-out calibration intercept/slope and grouped pairwise log-RR estimates. Work on large groups because the H and L side-arm buyer counts are the limiting information.

HTE calibration research emphasizes that discrimination and calibration are distinct; a model can rank effects but exaggerate their magnitude [2].

5.4 Ranking and prioritization

For “best places to adjust,” report top-fraction curves. RATE/TOC methods provide a general framework for evaluating whether a score prioritizes units with larger treatment effects [3]. In this application, pairwise likelihood and contribution-policy evaluation remain primary because RATE is naturally expressed on risk-difference or reward scales, while the requested estimand is a risk ratio.

5.5 Policy value

Evaluate the pricing rule by inverse propensity scoring on all held-out eligible quotes. The randomization probabilities are known, so no conversion model is required. Compare control, uniform H, uniform L, global scalar, flexible models, and selective top-k policies. Use customer-cluster bootstrap confidence intervals.

Doubly robust policy evaluation can be an optional variance-reduction check when a conversion/reward model is available [4], but it should not replace the model-free IPS benchmark.

5.6 Stability and temporal validity

A model must survive:

- outer group folds;
- multiple seeds;
- rolling forward-time tests;
- tariff-version and channel slices;
- top-k set overlap;
- missingness and out-of-support checks.

Small NLL improvements that do not reproduce in time should be treated as noise.

6. Cross-validation and leakage control

Use a final group-disjoint chronological holdout and nested group CV in development. `StratifiedGroupKFold` attempts to preserve class proportions while keeping groups non-overlapping [5]. Because the target among buyers is arm, stratify buyer folds by H/C/L and verify the side-arm count in every fold.

Calibration is itself a fitted model. Generate inner OOF raw scores, fit the calibrator on those scores, refit the base learner on the full outer-training set, and evaluate calibrated predictions on the outer fold.

All preprocessing is fold-local. This includes imputation, rare-level grouping, one-hot category maps, scalers, feature selection, and interaction construction.

7. Optuna design

Optuna’s multivariate/group TPE can model dependencies in conditional spaces, while the median pruner can terminate weak trials after comparable intermediate fold results [15–16]. Use persistent RDB storage and one study per model family and contrast.

The search objective is proper conditional NLL. Policy value is too variable for inner-loop optimization and can induce severe winner’s-curse bias. Use policy value only after proper-score and calibration gates.

7.1 Hyperparameters with highest value

The most important controls are:

1. minimum observations/minority buyers per leaf;

2. minimum Hessian per leaf or child;
3. depth/leaves;
4. L2 regularization;
5. learning rate and early-stopped rounds;
6. row/feature subsampling;
7. calibration slope.

Random-Forest tree count should normally be fixed once predictions stabilize. Random seeds are not hyperparameters.

7.2 Search-space summary

LightGBM pairwise

- learning rate: 0.005–0.10 log;
- depth: 2–7;
- leaves: $4 - \min(64, 2^{\text{depth}-1})$;
- dynamic `min_data_in_leaf` to retain roughly 10–100 side-arm buyers per leaf;
- `min_sum_hessian_in_leaf`: 0.1–50 log;
- feature fraction: 0.5–1;
- bagging fraction: 0.6–1;
- L1: $1e-8$ –100;
- L2: $1e-3$ –100;
- split gain: zero or $1e-5$ –1;
- max bin: 63, 127, 255, 511.

XGBoost pairwise

- eta: 0.005–0.10 log;
- depth: 2–7 or lossguide leaves 8–64;
- child weight: 1–100 log;
- subsample: 0.6–1;
- column sample: 0.5–1;
- gamma: zero or $1e-6$ –10;
- alpha: $1e-8$ –100;
- lambda: $1e-3$ –100;
- max bin: 64, 128, 256, 512;
- max delta step: 0, 1, 2, 5;
- scale positive weight fixed to one.

Random Forest pairwise

- trees fixed around 1,000;
- criterion: log loss or Gini;
- depth: 4–16;
- dynamic minimum leaf;
- max features: sqrt, log2, or 0.2–1;
- max samples: 0.5–1;
- impurity decrease: zero or $1e-8$ – $1e-3$;
- `ccp_alpha`: zero or $1e-8$ – $1e-2$;
- class weight fixed to none.

Scalar custom boosting

Use depth 1–4, learning rate 0.005–0.05, strong regularization, and much smaller Hessian thresholds than binary logistic boosting. The small price-dose variance changes the curvature scale.

Detailed spaces and executable suggest functions are in `elasticity_only/optuna_spaces.py`.

8. Feature interpretation

Use held-out permutation importance with pairwise NLL. scikit-learn notes that held-out permutation importance measures contribution to generalization and avoids the high-cardinality bias of tree impurity importance, while correlated predictors still require caution [9].

Interpret importance as predictive of price-response variation, not as a causal effect of the feature. The price arm is randomized; customer characteristics are not.

Priority features for investigation include base premium, margin, tariff uncertainty, competitor gap, channel, renewal status, and tariff version. Interactions that do not reproduce across folds or time should be removed even when a tree repeatedly selects them in-sample.

9. Decision gates and expected practical winner

The likely practical front-runners are:

- the global scalar model when heterogeneity signal is weak;
- calibrated pairwise LightGBM or XGBoost when stable nonlinear heterogeneity exists;
- a convex ensemble that shrinks those models toward the global anchor.

Random Forest is most valuable as an independent benchmark and a source of ensemble diversity. It should not be favored because it is “nonparametric”; with limited side-arm buyers, its flexibility can be a liability.

Select in this order:

1. validate the experiment;
2. establish the global response;
3. require OOF NLL improvement over global;
4. require calibration and low projection burden;
5. require time and seed stability;
6. choose among eligible models by lower confidence bound on IPS contribution lift;
7. prefer the simpler model if intervals materially overlap.

10. What not to do

The principal dead ends are:

- synthetic model tournaments;
- class balancing without exact correction;
- SMOTE or duplicated buyers;
- one-versus-rest probabilities treated as coherent multiclass probabilities;
- using displayed experimental price as a feature;
- row-random CV with repeat customers;
- tuning AUC/VUS or accuracy;
- estimating calibration on training data;
- selecting a policy from plug-in predicted value without randomized evaluation;
- claiming individual true elasticity from three-arm data;
- forcing a highly flexible two-gap surface before the scalar model is beaten.

11. Delivered implementation

The accompanying package contains:

- exact Bayes and relative-contribution identities;
- schema and experiment audits;
- group/time splitting;
- fold-safe preprocessing;
- global scalar and global pairwise fits;

- propensity-offset LightGBM and XGBoost pairwise models;
- inverse-propensity scikit-learn and Random-Forest pairwise wrappers;
- scalar custom objectives for LightGBM and XGBoost;
- Random-Forest moment inversion;
- multiclass benchmarks;
- OOF affine calibration and monotonic projection;
- likelihood, calibration, stability, and policy metrics;
- IPS/SNIPS and cluster bootstrap policy evaluation;
- Optuna search-space and nested-CV helpers;
- convex stacking;
- real-data-only examples and configuration.

The unit tests use deterministic toy arrays only to validate identities, gradients, offsets, and APIs. They do not produce or support a model ranking.

12. Bottom line

An elasticity-only model is coherent for the stated business objective because base conversion cancels from the quote-level choice among tested prices. The buyer Bayes-flip likelihood is the correct primary target for relative risk. The strongest production research design is:

global scalar anchor
 + calibrated pairwise LightGBM/XGBoost challengers
 + heavily smoothed Random-Forest benchmark
 + constrained OOF ensemble
 + proper conditional likelihood selection
 + randomized IPS contribution evaluation
 + group/time validation.

The data, not the algorithm brand, must decide whether personalization is justified. A finding that the global elasticity model cannot be beaten is a successful and commercially useful outcome, not a modeling failure.

Part II — LLM agent operating instruction

0. Role, mission, and evidence standard

You are an applied machine-learning researcher and production Python engineer. Your task is to use a **real randomized motor-insurance price test** to learn where the insurer can safely raise or lower price. Focus on **relative demand, finite-difference price elasticity, and pricing decisions among the tested arms**. Do not make base-conversion prediction the main task.

The experiment has three arms:

- **H**: approximately +10% price;
- **C**: control price;
- **L**: approximately -10% price.

The nominal assignment may be 10/80/10, but always use the **logged row-level assignment probabilities** π_H , π_C , and π_L .

Your evidence standard is strict:

1. Every empirical result must come from the supplied real randomized quote table.
2. Every reported candidate-model score must be strictly out of fold or on a final untouched chronological holdout.
3. Synthetic observations may be used only for deterministic unit tests of code or derivatives. Never use generated data to select, rank, or recommend a model for the insurer.
4. Never report individual elasticity RMSE, individual treatment-effect correlation, or “ground-truth” customer elasticity on real data; these targets are unobserved.
5. Preserve positive and negative findings, failed models, warnings, and dead ends in the research log.
6. If the experiment table or required fields are missing, stop and produce a missing-data report. Do not invent data and do not substitute a public observational dataset.

1. Core estimand

For pre-treatment customer and vehicle features $X=x$, define

- $T \in \{H, C, L\}$ as randomized price arm;
- $Y \in \{0, 1\}$ as purchase;
- $\pi_t(x) = P(T=t \mid X=x)$ as the known randomized propensity;
- $q_t(x) = P(Y=1 \mid T=t, X=x)$ as arm-specific quote conversion;
- $r_t(x) = P(T=t \mid Y=1, X=x)$ as the buyer-side arm posterior.

Use the exact Bayes identity

$$r_t(x) = \pi_t(x) q_t(x) / \sum_j \pi_j(x) q_j(x).$$

The relative demand to control is identified by

$$\begin{aligned} RR_{tC}(x) &= q_t(x) / q_C(x) \\ &= [r_t(x) / \pi_t(x)] / [r_C(x) / \pi_C(x)]. \end{aligned}$$

For a fixed 10/80/10 assignment:

$$\begin{aligned} RR_{HC}(x) &= 8 r_H(x) / r_C(x) \\ RR_{LC}(x) &= 8 r_L(x) / r_C(x). \end{aligned}$$

Demand monotonicity means

$$RR_{HC}(x) \leq 1 \leq RR_{LC}(x),$$

or equivalently

$$r_H(x)/\pi_H(x) \leq r_C(x)/\pi_C(x) \leq r_L(x)/\pi_L(x).$$

The adjacent finite-difference elasticity magnitudes are

$$\begin{aligned}\beta_H(x) &= -\log RR_{HC}(x) / \log(1.10) \\ \beta_L(x) &= -\log RR_{LC}(x) / \log(0.90).\end{aligned}$$

Because $\log(0.90) < 0$, both magnitudes are positive under decreasing demand.

Do not silently call these derivatives at a point. They are finite-difference elasticities over the tested intervals. The scalar model below adds a local constant-elasticity assumption.

2. Why a base-conversion model is not required for the price choice

Let $g_t(x)$ be expected contribution if the policy is purchased at arm t . It must include the business-relevant arm-specific premium, expected claims, commissions, taxes, payment expenses, and other variable costs.

Expected contribution per quote is

$$\begin{aligned}V_t(x) &= g_t(x) q_t(x) \\ &= q_C(x) g_t(x) RR_{tC}(x).\end{aligned}$$

The positive common factor $q_C(x)$ cancels when choosing among arms:

$$d^*(x) = \operatorname{argmax}_t g_t(x) RR_{tC}(x), \text{ with } RR_{CC}(x)=1.$$

Therefore, the primary model can be elasticity-only. Report the relative contribution factor

$$B_t(x) = [g_t(x)/g_C(x)] RR_{tC}(x)$$

and relative lift $B_t(x)-1$.

A base-conversion model is optional and secondary. It may be useful for variance reduction in doubly robust policy evaluation, total-volume forecasting, or covariate-shift weighting. It must not dominate model training or model selection.

3. Required inputs and immediate stop conditions

Read `DATA_CONTRACT.md`. At minimum require one row per eligible randomized quote with:

- unique `quote_id`;
- stable `customer_id` or household cluster;
- `quote_timestamp`;
- `eligible`;
- `arm`;
- `purchase`;
- `pi_H`, `pi_C`, `pi_L`;
- explicit pre-treatment feature list.

For business policy evaluation also require `contribution_H`, `contribution_C`, and `contribution_L`.

Stop model fitting and issue an error report if any of the following occurs:

- assignment propensities are missing, nonpositive, or do not sum to one;
- an arm has no eligible quotes or no buyers;
- assignment was not randomized or the randomization mechanism cannot be reconstructed;
- eligibility or rows were removed after seeing arm or outcome;
- feature timing is unknown;

- customer grouping is absent despite repeated quotes;
- the final outcome window is undefined;
- the experiment implementation changed without a corresponding block, propensity, or version field.

4. Reproducible project initialization

Create a run directory containing:

```
run_<UTC timestamp>_<git or code hash>/
  config_resolved.yaml
  environment.txt
  data_fingerprint.json
  audit/
  folds/
  optuna/
  models/
  oof_predictions/
  holdout_predictions/
  metrics/
  plots/
  research_log.md
  model_cards/
```

Record:

- SHA-256 hash of the exact input file bytes;
- row count and column names;
- package versions;
- random seeds;
- code commit or source hash;
- resolved configuration;
- every data exclusion and its reason.

Never overwrite a completed run directory.

5. Phase A: audit the randomized experiment before machine learning

5.1 Population accounting

Produce a CONSORT-like flow table:

1. all records received;
2. eligible randomized quotes;
3. outcomes observed within the defined window;
4. buyers and nonbuyers by arm;
5. repeated customers and repeated quote opportunities;
6. rows excluded, with pre-specified reasons.

Report arm-level quote and buyer counts. Use side-arm buyer counts, not total quote count, when judging heterogeneity capacity.

5.2 Assignment audit

For each arm, compare observed assignment count with the sum of logged propensities. Report standardized residuals and inspect them by:

- calendar week or day;
- randomization block;

- tariff version;
- channel;
- geography;
- device or acquisition source;
- base-premium band.

Unexpected residual patterns are implementation issues, not modeling opportunities.

5.3 Covariate balance

On all eligible quotes, compare pre-treatment covariates across H/C and L/C. Report standardized mean differences for numeric variables and distribution differences for categorical variables. Large systematic imbalance indicates an assignment or logging problem. Do not “fix” genuine randomization imbalance by propensity modeling unless the logged design probabilities are wrong; use the known randomized propensities.

5.4 Price-display compliance

Compare displayed premium divided by base premium with the assigned multiplier. Quantify rounding, minimum premiums, fees, installments, manual overrides, and failed displays. The primary causal analysis is intention-to-treat by randomized arm. A realized-dose analysis is secondary and should be treated as an instrumental-variable or compliance problem, not as a replacement for the randomized assignment.

5.5 Outcome integrity

Verify that purchase is measured identically across arms and that the window is not truncated by time. Perform pre-specified sensitivity analyses for early cancellation or payment failure only if those outcomes are consistently available.

5.6 Leakage audit

The model feature list must be explicit. Reject:

- arm and assigned multiplier;
- displayed experimental price;
- any variable computed after assignment using the displayed price;
- purchase, policy ID, payment choice, cancellation, or claims observed after purchase;
- agent or customer actions that happened after seeing price.

Pre-randomization base premium, expected cost, tariff version, uncertainty, and competitor position may be used.

6. Phase B: define folds once and reuse them for every model

6.1 Final chronological holdout

Reserve the most recent 15–25% of customer or household clusters as a final holdout. Assign a cluster to the holdout using its latest eligible quote time, so no cluster appears in both development and holdout data. Do not inspect holdout model rankings during Optuna tuning.

6.2 Nested development cross-validation

Within the development period:

- use 5 outer group-disjoint folds for model comparison;
- use 4 inner group-disjoint folds for Optuna;
- stratify buyer folds by arm using `StratifiedGroupKFold` when feasible;
- verify every validation fold contains enough H/C and L/C buyers;
- use the exact same outer folds for every candidate.

Also run 2–4 rolling forward-time backtests. A model that wins random group CV but fails forward-time validation is not production eligible.

6.3 Calibration splits

Calibration must use predictions from observations that were not used to train the base model. For each outer fold:

1. generate inner out-of-fold raw scores on the outer-training set;
2. fit the risk-ratio calibrator on those inner OOF scores;
3. refit the base learner on all outer-training buyers;
4. apply the learned calibrator to the outer-validation raw scores.

Never fit a probability or elasticity calibrator on in-sample predictions.

7. Feature preparation

Use only pre-treatment features. Preserve business meaning and prefer stable features over opaque identifiers.

7.1 Candidate feature groups

Consider:

- base premium and log base premium;
- expected claim cost and intended margin;
- tariff uncertainty, model version, and out-of-distribution score;
- customer, driver, vehicle, geography, and coverage attributes;
- channel and acquisition context;
- new business versus renewal;
- pre-treatment competitor gap or market rank;
- quote-time seasonality and market conditions;
- historical customer behavior known before this quote.

Do not automatically include every available field.

7.2 Fold-local preprocessing

Fit imputers, category maps, encoders, scalers, and feature-selection steps inside each training fold only.

- For linear models: median-impute numeric features, add missing indicators, standardize, and one-hot encode categories.
- For Random Forest: impute numeric variables and use fold-local one-hot or ordinal encoding. Avoid target encoding based on randomized arm.
- For LightGBM: native categorical columns are acceptable if category levels are fixed from the training fold. One-hot encoding is a valid portability baseline.
- For XGBoost: use one-hot encoding by default for portability. Native categorical handling may be used only with version-controlled serialization and fold-local category handling.

Combine rare categorical levels using a minimum frequency chosen before or inside inner CV. Do not use buyer arm as a target encoder because randomization noise will be amplified.

8. Mandatory model baselines

Fit these before high-capacity models.

8.1 No-effect model

Use

$$r_t(x) = \pi_t(x), \text{RR}_{\text{HC}} = \text{RR}_{\text{LC}} = 1.$$

This is the null benchmark for buyer log loss and information gain.

8.2 Global unrestricted two-gap model

Fit constant $\log RR_{HC}$ and $\log RR_{LC}$ by the pairwise propensity-offset likelihoods. Fit both unconstrained and monotone-constrained versions. This estimates the portfolio-level finite-difference response without heterogeneity.

8.3 Global scalar elasticity model

Fit a single nonnegative β :

$$\begin{aligned} r_t(x) &\text{proportional to } \pi_t(x) \exp[-\beta z_t], \\ z_t &= \log(P_t/P_C). \end{aligned}$$

This is the most important anchor. A flexible model must demonstrate real OOF improvement over it. If it does not, deploy the global model or do not personalize.

8.4 Penalized pairwise logistic model

Fit H/C and L/C separately using inverse-propensity buyer weights or a fixed offset implementation. Include main effects and a small, pre-specified interaction set. This is the interpretable heterogeneous benchmark.

9. Main model family: pairwise buyer likelihoods

For contrast t versus control among buyers:

$$\begin{aligned} P(T=t \mid T \in \{t, C\}, Y=1, X=x) \\ = \pi_t(x) q_t(x) / [\pi_t(x)q_t(x) + \pi_C(x)q_C(x)]. \end{aligned}$$

Therefore

$$\begin{aligned} \text{logit } P(T=t \mid \text{pair, buyer, } X=x) \\ = \log[\pi_t(x)/\pi_C(x)] + \log RR_{tC}(x). \end{aligned}$$

Fit two models:

- H versus C, estimating $\log RR_{HC}(x)$;
- L versus C, estimating $\log RR_{LC}(x)$.

Reconstruct a coherent three-arm buyer posterior:

$$r(x) = \text{softmax}(\log \pi(x) + [\log RR_{HC}(x), 0, \log RR_{LC}(x)]).$$

This reconstruction is mandatory. Do not treat two one-versus-rest probabilities as independent final probabilities.

9.1 XGBoost pairwise model

Use native binary logistic loss with the known row-specific propensity log odds as **base_margin**. The booster learns the residual $\log RR_{tC}(x)$. This is preferable to class balancing because it retains the exact conditional likelihood without increasing variance through inverse-propensity reweighting.

Set a global pairwise log-RR MLE as the initial residual intercept, then let trees learn heterogeneous deviations. Use `tree_method='hist'`, early stopping, shallow trees, large child weights, and strong regularization.

Do not set `scale_pos_weight` to buyer-class frequency. Keep it equal to one; the unequal buyer-arm frequency is part of the estimand.

9.2 LightGBM pairwise model

Use $\text{init_score} = \log(\pi_t/\pi_C) + \text{global_log_RR_tC}$ on the training and validation pairwise buyer rows. Set $\text{boost_from_average}=\text{False}$. The learned tree score is a heterogeneous deviation from the global log risk ratio. At prediction, add the stored global log RR to the tree contribution; the row-specific propensity offset is used only when converting the risk ratio into the buyer pair probability.

Use conservative num_leaves , max_depth , min_data_in_leaf , and $\text{min_sum_hessian_in_leaf}$. Leaf-wise growth can overfit weak interaction signal unless these controls are strong.

9.3 Random-Forest pairwise model

scikit-learn Random Forest has no fixed logit-offset API. On pairwise buyers, weight every row by

$$w_i = 1 / \pi_{\{T_i\}}(X_i).$$

In the weighted buyer population,

$$P_w(T=t \mid \text{pair, buyer, } X=x) = q_t(x) / [q_t(x) + q_C(x)],$$

so the predicted logit is $\log \text{RR_tC}(x)$.

Use $\text{class_weight}=\text{None}$. Automatic balanced class weights would change the target again. Fit large, heavily smoothed forests and calibrate the log-risk-ratio score out of fold.

Random Forest is a valuable robustness benchmark and nonlinear challenger, but its raw class probabilities are often too coarse for elasticity ratios. Calibration and large leaves are mandatory.

9.4 Penalized scikit-learn logistic model

Use the same inverse-propensity weighting as Random Forest. Use **Saga** for L2 or elastic-net regularization. Scale numeric inputs. Do not use automatic balanced class weights. Use this model to test whether complex trees add genuine heterogeneity beyond stable additive effects.

10. Scalar elasticity family

Assume local constant elasticity over the tested range:

$$\log \text{RR_tC}(x) = -\beta(x) [z_t - z_C], \beta(x) \geq 0.$$

The buyer NLL for row i is

$$L_i = \beta(x_i) z_{\{T_i\}} + \log \sum_j \pi_{ij} \exp[-\beta(x_i) z_{ij}].$$

Its derivative with respect to β is

$$z_{\{T_i\}} - E_r[z \mid x_i],$$

and its curvature in β is

$$\text{Var}_r(z \mid x_i) \geq 0.$$

10.1 XGBoost and LightGBM custom objectives

Let the raw tree score be $f(x)$ and set

$$\beta(x) = \text{softplus}(f(x)).$$

Use gradient

$$g_f = [z_T - E_r(z)] \text{sigmoid}(f).$$

Use a positive Fisher/Gauss-Newton Hessian

$$h_f = \text{Var}_r(z) \text{sigmoid}(f)^2 + \text{epsilon}.$$

Do not rely on the exact raw-score Hessian because the nonlinear positivity transform can make it negative. Custom tree objectives must be row-additive; the scalar model satisfies that requirement.

Because the tested log-price doses are only about +/-0.1, Fisher curvature per buyer is small. Use much smaller `min_child_weight` / `min_sum_hessian_in_leaf` ranges than ordinary binary logistic boosting, but compensate with shallow trees, large data leaves, and strong L2 regularization.

10.2 Random-Forest moment model

For a Random Forest scalar benchmark, inverse-propensity-weight buyers and regress the observed centered log-price dose on $\mathbf{X}$. Under the scalar exponential-family model, the weighted conditional mean dose is a one-to-one decreasing function of β . Invert that function by vectorized bisection.

Use squared-error regression only; absolute-error regression targets a median and is not valid for the mean-moment inversion. Treat this as a robust benchmark, not as a likelihood-equivalent estimator.

10.3 When to prefer scalar over two-gap models

Prefer scalar when:

- side-arm buyer counts are limited;
- +/-10% is a local experiment;
- H/C and L/C effects are statistically compatible;
- flexible two-gap models are unstable across folds or time;
- the scalar model has equal or better OOF likelihood.

Permit separate H/C and L/C surfaces only after they improve held-out likelihood and calibration. Apparent upper/lower asymmetry can arise from population mixing, noise, or different side-arm precision, not necessarily true individual curvature.

11. Multiclass models: benchmark, not automatic default

Train an inverse-propensity-weighted multiclass buyer classifier to estimate

$$s_t(x) = q_t(x) / \sum_j q_j(x).$$

Convert to risk ratios by S_t/S_C . Test multinomial logistic regression, Random Forest, LightGBM, and XGBoost.

Use multiclass models as benchmarks because:

- the control class dominates raw buyer frequency;
- the model spends capacity on a redundant common normalization;
- monotonicity is not automatic;
- pairwise models allow contrast-specific regularization and diagnostics.

If a multiclass model wins, project its log density scores onto the ordered cone, recalibrate, and quantify the fraction and magnitude of changed predictions.

12. Calibration, shrinkage, and constraints

Raw tree probabilities are not sufficient. Elasticity depends on odds and probability ratios, so calibration errors are amplified.

For each contrast, fit on inner OOF predictions:

$$\log \text{RR}_{\text{cal}}(x) = \alpha + s [\text{raw}(x) - \text{center}], \quad s \geq 0.$$

Estimate α and S in the exact pairwise buyer likelihood with the known propensity offset. Interpret:

- $S=0$: no reproducible heterogeneity; collapse to a global effect;
- $0 < S < 1$: raw model is over-dispersed and should be shrunk;
- $S \approx 1$: dispersion is approximately calibrated;
- $S > 1$: raw model is under-dispersed.

Enforce physical monotonicity after calibration:

```
log RR_HC <= 0
log RR_LC >= 0.
```

Hard clipping is transparent. A smooth softplus projection may be used during optimization. Always report:

- raw violation rate;
- average and maximum projection distance;
- percentage of predictions exactly on the boundary;
- score before and after projection.

A high projection rate is evidence of a weak or misspecified model, not proof that post-processing solved it.

Use isotonic calibration only when the calibration sample is sufficiently large and each contrast has ample side-arm buyers. Prefer affine/sigmoid calibration at smaller sample sizes.

13. Optuna protocol

13.1 General design

Create a separate Optuna study for each:

- contrast H/C and model family;
- contrast L/C and model family;
- scalar model family;
- multiclass benchmark family.

Do not place every family in one giant conditional search space. Separate studies give TPE enough comparable trials and preserve clear failure logs.

Use:

```
TPESampler(
    seed=SEED,
    n_startup_trials=30,
    multivariate=True,
    group=True,
)
MedianPruner(
    n_startup_trials=15,
    n_warmup_steps=1,
    n_min_trials=5,
)
```

Persist studies in SQLite or another RDB. Use `n_jobs=1` at the Optuna level and allocate threads inside the learner, or run distributed workers with each learner constrained to a known thread count. Avoid nested uncontrolled parallelism.

For each trial:

1. loop over the fixed inner group folds;
2. fit all preprocessing inside the fold;
3. use native early stopping for boosting;
4. evaluate pairwise conditional NLL or scalar buyer NLL;
5. call `trial.report(mean_completed_fold_score, fold_number)`;
6. prune only after at least one complete validation fold;
7. save all fold scores, best iteration counts, and warnings as trial attributes.

Do not optimize AUC, VUS, accuracy, F1, in-sample loss, or estimated policy value.

13.2 Trial budgets

Use these as starting budgets, not guarantees:

Family	Trials per contrast/model
Penalized logistic	40–80
Random Forest pairwise	80–150
LightGBM pairwise	150–250
XGBoost pairwise	150–250
Random-Forest moment scalar	60–120
LightGBM scalar custom	100–200
XGBoost scalar custom	100–200
Multiclass benchmark	80–150

After choosing hyperparameters, refit the top configurations over 3–5 fixed random seeds. Do not tune the random seed.

13.3 Dynamic minimum leaf rule

Let n_{pair} be the number of buyers in a contrast and ρ the side-arm fraction within that pair. Set the minimum search bound to contain approximately 10 minority-arm buyers per leaf:

```
leaf_lower = max(20, ceil(10/rho)).
```

Set the upper bound to the smaller of 25% of pair buyers and about 100 minority-arm buyers:

```
leaf_upper = max(leaf_lower,
                  min(floor(0.25*n_pair), ceil(100/rho))).
```

If this leaves room for only a few leaves, that is correct evidence that rich heterogeneity is not supported.

13.4 LightGBM pairwise search space

Optimize:

Parameter	Range	Notes
<code>learning_rate</code>	0.005–0.10 log	Use up to 5,000 rounds with early stopping.
<code>max_depth</code>	2–7	Weak signal favors shallow trees.
<code>num_leaves</code>	4 to $\min(64, 2^{\text{depth}-1})$	Respect depth.
<code>min_data_in_leaf</code>	dynamic log-int	Highest-impact variance control.
<code>min_sum_hessian_in_leaf</code>	0.1–50 log	Important for pairwise logistic curvature.
<code>feature_fraction</code>	0.5–1.0	Feature subsampling.
<code>bagging_fraction</code>	0.6–1.0	Row subsampling.

Parameter	Range	Notes
bagging_freq	1-10	Use when fraction <1.
lambda_l1	1e-8-100 log	Include zero as separate categorical branch if desired.
lambda_l2	1e-3-100 log	Usually important.
min_gain_to_split	0 or 1e-5-1 log	Conditional zero/positive branch.
max_bin	63, 127, 255, 511	Lower values regularize and speed training.

Fix `boosting_type='gbdt'`, `boost_from_average=False`, and `feature_pre_filter=False` during tuning. Test DART only as a separate late-stage study because its early-stopping and calibration behavior differ.

13.5 XGBoost pairwise search space

Optimize:

Parameter	Range	Notes
eta	0.005-0.10 log	Up to 5,000 rounds with early stopping.
grow_policy	depthwise, lossguide	Conditional space.
max_depth	2-7	For depthwise.
max_leaves	8-64 log-int	For lossguide.
min_child_weight	1-100 log	Main smoothing control.
subsample	0.6-1.0	Row subsampling.
colsample_bytree	0.5-1.0	Feature subsampling.
gamma	0 or 1e-6-10 log	Split-gain threshold.
reg_alpha	1e-8-100 log	L1.
reg_lambda	1e-3-100 log	L2.
max_bin	64, 128, 256, 512	Histogram resolution.
max_delta_step	0, 1, 2, 5	Can stabilize imbalanced logistic updates.

Fix `tree_method='hist'` and `scale_pos_weight=1`.

13.6 Random-Forest pairwise search space

Set `n_estimators=1000` initially. Verify that OOB or held-out score has stabilized; then do not waste Optuna trials tuning tree count.

Optimize:

Parameter	Range	Notes
criterion	log_loss, gini	Log-loss is natural, but test both.
max_depth	4-16	Large leaves remain the main guardrail.
min_samples_leaf	dynamic log-int	Most important.
max_features	sqrt, log2, or 0.2-1.0	Conditional branch.
max_samples	0.5-1.0	Requires bootstrap.
min_impurity_decrease	0 or 1e-8-1e-3 log	Additional pruning.
ccp_alpha	0 or 1e-8-1e-2 log	Cost-complexity pruning.

Fix `bootstrap=True`, `class_weight=None`, and use inverse-propensity `sample_weight`.

13.7 Penalized logistic search space

Optimize:

- C: 1e-4-100 log;
- L2 versus elastic net;
- l1_ratio: 0-1 for elastic net;
- optional interaction set as a small categorical structural choice.

Use `saga`, `max_iter` ≥ 5000, scaled numeric features, and no class weights.

13.8 Scalar custom-objective search spaces

For XGBoost:

- eta: 0.005-0.05 log;
- max_depth: 1-4;
- min_child_weight: 1e-4-1 log;
- subsample: 0.6-1;
- colsample_bytree: 0.5-1;
- gamma: 1e-7-1 log;
- reg_alpha: 1e-8-100 log;
- reg_lambda: 1e-3-100 log;
- max_bin: 64, 128, 256.

For LightGBM:

- learning_rate: 0.005-0.05 log;
- max_depth: 1-4;
- num_leaves: 2-min(15, 2^{depth});
- min_data_in_leaf: 1%-20% of buyers, log-int, with a minimum of 30;
- min_sum_hessian_in_leaf: 1e-5-0.5 log;
- feature_fraction: 0.5-1;
- bagging_fraction: 0.6-1;
- lambda_l1: 1e-8-100 log;
- lambda_l2: 1e-3-100 log.

The small Hessian ranges are specific to the price-dose curvature. Do not copy ordinary binary-logistic defaults blindly.

13.9 Random-Forest moment search space

Fix `criterion='squared_error'` and approximately 1,000 trees. Optimize:

- max_depth: 3-12;
- min_samples_leaf: 0.01-0.15 of buyers, log;
- max_features: 0.3-1;
- max_samples: 0.6-1;
- ccp_alpha: 1e-9-1e-2 log;
- beta_max: 10, 20, 30 as a sensitivity bound.

14. Hyperparameters that matter most

Prioritize tuning effort in this order:

1. leaf/child sample and Hessian constraints;
2. tree depth or leaf count;
3. L2 regularization;
4. learning rate with early-stopped round count;
5. row and feature subsampling;
6. L1 and split-gain thresholds;
7. calibration slope and global shrinkage.

Do not spend major search budget on:

- random seed;
- `n_estimators` for Random Forest after convergence;
- cosmetic logging parameters;
- class weights that alter the target;
- extremely deep trees that the leaf constraints will prevent anyway.

15. Primary model-quality metrics

15.1 Buyer multiclass NLL

For coherent posterior $\mathbf{r}_{\text{hat}}$:

$$\text{NLL}_{\text{buyer}} = -\text{mean}_{\{\text{buyers}\}} \log \mathbf{r}_{\text{hat}}\{\mathbf{T}_i\}(\mathbf{X}_i).$$

This is a proper score for the flip posterior.

15.2 Pairwise conditional NLL

For contrast $\mathbf{t}/\mathbf{C}$:

$$\begin{aligned} \eta_{\mathbf{i}} &= \log(\pi_{\mathbf{i}\mathbf{t}}/\pi_{\mathbf{i}\mathbf{C}}) + \log \text{RR}_{\text{hat}}_{\mathbf{t}\mathbf{C}}(\mathbf{X}_{\mathbf{i}}) \\ \text{NLL}_{\mathbf{t}\mathbf{C}} &= \text{mean}_{\{\text{pair buyers}\}} [\log(1 + \exp \eta_{\mathbf{i}}) - y_{\mathbf{i}} \eta_{\mathbf{i}}]. \end{aligned}$$

Use

$$\text{NLL}_{\text{equal}} = 0.5 \text{NLL}_{\text{HC}} + 0.5 \text{NLL}_{\text{LC}}$$

as the main cross-family metric. It prevents the 80% control arm and unequal pair sizes from determining the result by themselves.

15.3 Elasticity information gain

Within each outer fold, compare a heterogeneous model with the fitted global pairwise model:

$$\text{IG}_{\mathbf{t}} = \text{NLL}_{\text{global},\mathbf{t}} - \text{NLL}_{\text{model},\mathbf{t}}.$$

Report millinats per pair buyer and a customer-clustered paired confidence interval. The heterogeneous model is not credible if the interval includes zero across development CV and the forward holdout.

15.4 Calibration

For each contrast, evaluate on held-out predictions:

$$\begin{aligned} &\text{logit } P(\mathbf{T}=\mathbf{t} \mid \text{pair}, \text{buyer}, \mathbf{X}) \\ &= \log(\pi_{\mathbf{t}}/\pi_{\mathbf{C}}) + \mathbf{a} + \mathbf{b} * \text{predicted}_{\log_RR}. \end{aligned}$$

Report calibration intercept and nonnegative slope. Also create 4–5 large groups by predicted score and fit an intercept-only log RR within each group using the propensity offset.

Do not create ten small deciles when side-arm buyer counts are low.

15.5 Ranking and targeting

Use top-fraction curves and, where implemented with valid inference, RATE/TOC-style metrics to test whether highly ranked quotes show larger randomized price response. Ranking metrics are supplementary: they do not replace probability calibration or pairwise NLL.

VUS, multiclass AUC, and ordinary accuracy are secondary diagnostics only. They do not validate the numerical odds ratios needed for elasticity.

15.6 Stability

Report:

- fold-to-fold metric standard deviation;
- seed-to-seed prediction correlation;
- Spearman correlation of elasticity scores;
- Jaccard overlap of top 5%, 10%, and 20% adjustment sets;
- calibration and information gain by time period;
- performance by channel and tariff version;
- extrapolation and missingness sensitivity.

16. Business policy evaluation without a conversion model

Construct the three-arm policy

$$\text{policy}(x) = \operatorname{argmax}_t \text{contribution}_t(x) * \text{RR_hat}_{tC}(x).$$

Also construct selective policies that adjust only the top 5%, 10%, 20%, 40%, 60%, or 100% of quotes by predicted positive relative contribution gain, leaving the rest at control.

On strictly held-out eligible quotes, estimate policy contribution with inverse propensity scoring:

$$V_{\text{IPS}}(d) = (1/N) \sum_i I[T_i=d(X_i)] Y_i g_{\{T_i\}}(X_i) / \pi_{\{i,T_i\}}.$$

Also report the self-normalized/Hajek version. Compare:

- control for everyone;
- each uniform tested arm;
- global scalar policy;
- each calibrated flexible model;
- the constrained ensemble;
- selective adjustment policies.

Use customer-cluster bootstrap confidence intervals for paired policy differences. Choose business thresholds on inner folds and evaluate them on outer folds. The final chronological holdout is used once for confirmation.

An optional base-conversion model may support a doubly robust value estimator, but IPS remains the required model-free benchmark because the randomization propensities are known.

17. Ensembling

After generating OOF log-risk-ratio predictions from eligible candidates, fit a convex stack separately for H/C and L/C:

$$\begin{aligned} \log \text{RR}_{\text{stack}} &= \sum_k w_k \log \text{RR}_k, \\ w_k &\geq 0, \sum_k w_k = 1. \end{aligned}$$

Optimize weights on OOF pairwise NLL, then recalibrate and impose sign constraints. A convex combination of constrained log RRs preserves the correct sign.

Do not stack in-sample predictions. Keep the global scalar model in the library so the stack can shrink toward it.

18. Interpretation and feature importance

Use held-out permutation importance with **pairwise conditional NLL** as the scoring rule. Do not rely on tree impurity importance; it is biased toward high-cardinality variables and can describe overfitting.

For the top stable features:

- plot partial dependence or accumulated local behavior for calibrated log RR, not arm probability;
- stratify by tariff version and time;
- check that the relationship is operationally plausible;
- inspect interaction stability across folds and seeds;
- record whether the feature is approved for pricing use.

Never interpret a split or SHAP contribution as a causal effect of that feature. Randomization identifies the price-arm effect conditional on features; it does not randomize customer characteristics.

19. Model-selection gates

Use a gated rather than single-number selection rule.

Gate 1: experiment validity

Pass assignment, eligibility, outcome, leakage, and compliance audits.

Gate 2: global signal

The global pairwise/scalar model must show a stable nonzero price response. If not, report insufficient evidence and do not fit highly personalized policies.

Gate 3: proper-score improvement

A flexible model must improve equal-contrast NLL over the global anchor with a paired customer-clustered confidence interval and confirm in forward-time testing.

Gate 4: calibration

The model must have acceptable calibration after OOF affine shrinkage, low projection burden, and sensible grouped response ordering.

Gate 5: stability

Top adjustment sets and score rankings must be stable across folds, seeds, time, and major operational segments.

Gate 6: business value

Among models passing the previous gates, choose the one with the highest **lower confidence bound** on held-out contribution lift relative to control or the best uniform tested arm. Prefer the simpler model when confidence intervals overlap materially.

No empirical winner may be declared until all gates are evaluated on real data.

20. Final refit and deployment artifacts

After final model choice:

1. freeze feature definitions, transformations, hyperparameters, calibrators, and constraints;
2. refit on the full development period only;
3. evaluate once on the final chronological holdout;
4. if accepted, refit on all approved data for deployment while preserving the frozen specification;
5. save model, preprocessor, calibration objects, contribution-policy logic, schema, package versions, and training-data hash;
6. build runtime checks for missing categories, out-of-range values, tariff version, and propensity/design changes;
7. produce an abstention flag when a quote is outside training support or model disagreement is high.

Monitor after deployment:

- randomized shadow-test buyer NLL;
- global and grouped risk ratios;
- adjustment frequency;
- policy IPS value;
- calibration drift;
- feature and tariff-version drift;
- constraint/projection rates.

Retain a continuing randomized control sample. A fully deterministic personalized policy removes the data needed to learn future elasticity.

21. Required outputs from the agent

Produce at least:

1. `data_validation_issues.csv`;
2. `experiment_summary.csv`;
3. assignment, balance, and dose-compliance audits;
4. immutable fold assignment file;
5. Optuna database and trial tables for every family;
6. OOF predictions for every candidate with `quote_id`, fold, raw and calibrated log RRs, elasticities, constraint flags, and policy arm;
7. final holdout predictions;
8. proper-loss, calibration, stability, and policy-value tables;
9. selective policy curves;
10. customer-cluster bootstrap intervals;
11. held-out permutation importance;
12. one model card per candidate;
13. a model-comparison decision log;
14. `research_log.md` listing positive findings, negative findings, failures, and unresolved risks;
15. reproducibility manifest and SHA-256 hashes.

22. Prohibited shortcuts and known dead ends

Do not:

- train only on buyers and then claim absolute conversion probability;
- balance buyer arms to one third without exact prior/weight correction;
- use `class_weight='balanced'` and interpret the resulting probabilities as risk ratios;
- oversample or SMOTE buyer arms as if this created experimental information;
- include displayed experimental price as a feature when arm is the target;
- tune on AUC, VUS, accuracy, or policy value alone;
- compare models on different folds;
- calibrate on training predictions;
- use row-random splits when customers repeat;
- average individual elasticities and call the result a marginal group elasticity without checking the aggregation;
- infer true individual elasticity from one observed arm;
- claim that a flexible tree is better because it has lower training loss;
- select a model from synthetic data;
- hide failed trials or constraint violations.

23. Recommended final candidate set

Unless the real data indicate otherwise, carry these candidates to outer validation:

1. no-effect null;
2. global unrestricted two-gap model;
3. global scalar elasticity model;
4. penalized pairwise logistic;
5. calibrated pairwise Random Forest;
6. calibrated pairwise LightGBM with propensity offsets;
7. calibrated pairwise XGBoost with propensity offsets;
8. Random-Forest scalar moment model;
9. LightGBM scalar custom objective;
10. XGBoost scalar custom objective;
11. inverse-propensity multiclass benchmarks;
12. convex OOF ensemble including the global anchor.

The expected practical front-runners are the global scalar anchor and calibrated pairwise LightGBM/XGBoost models. Random Forest is the principal robustness benchmark. This is a research prior, not an empirical conclusion; let the real nested OOF and chronological results decide.

24. Completion checklist

Before declaring the task complete, answer every item with evidence:

- ☐ Did the input pass the real randomized-data contract?
- ☐ Were all exclusions pre-treatment and documented?
- ☐ Are folds customer/household disjoint?
- ☐ Is there an untouched chronological holdout?
- ☐ Did every model use identical outer folds?
- ☐ Were preprocessing and calibration fold-local?
- ☐ Were logged row-level propensities used exactly?
- ☐ Were class balancing and oversampling avoided or exactly corrected?
- ☐ Were global scalar and global two-gap anchors fitted?
- ☐ Did Optuna optimize proper buyer likelihoods?
- ☐ Were native early stopping and pruning kept inside inner CV?
- ☐ Were probabilities calibrated out of fold?
- ☐ Were monotonicity violations and projection burden reported?
- ☐ Was heterogeneous improvement tested against the global model?
- ☐ Were forward-time results reported?
- ☐ Was the pricing policy evaluated by randomized IPS/SNIPS?
- ☐ Were uncertainty and cluster dependence handled?
- ☐ Were top-k adjustment sets stable?
- ☐ Were negative findings and dead ends retained?
- ☐ Were no synthetic model rankings used?
- ☐ Is the final recommendation conditional on real held-out evidence?

Use the functions and examples in the accompanying `elasticity_only` Python package. Keep all model-specific code behind a common interface that produces calibrated `log_rr_H`, `log_rr_L`, coherent buyer posteriors, finite-difference elasticities, and arm-specific relative contribution scores.

Part III — Real-data contract

This package is deliberately **real-data only**. It does not generate insurance observations and it must not substitute an unrelated public dataset when the randomized experiment table is missing.

1. Unit of analysis

Use one row per **eligible randomized quote opportunity**. Include buyers and nonbuyers, even though the elasticity-shape models are trained on buyers. The complete eligible table is required for experiment auditing and held-out policy evaluation.

The unit must correspond to the point at which the randomized price arm was assigned. If a customer can request repeated quotes, preserve all legitimate opportunities but provide a stable customer or household identifier so that folds, bootstrap samples, and holdouts remain group-disjoint.

2. Required columns for elasticity shape

Column	Type	Meaning
quote_id	string	Unique identifier for the randomized quote opportunity.
customer_id	string	Stable customer or household cluster identifier.
quote_timestamp	timestamp	Time at which the quote entered the experiment. Store timezone explicitly.
eligible	0/1	Whether the row was in the experiment's eligible population before assignment.
arm	H, C, L	Assigned high-price, control, or low-price arm.
purchase	0/1	Conversion within the pre-specified outcome window.
pi_H	float	Logged probability of assignment to H for this row.
pi_C	float	Logged probability of assignment to C for this row.
pi_L	float	Logged probability of assignment to L for this row.

Every propensity must be strictly positive and each row must satisfy

$$pi_H + pi_C + pi_L = 1.$$

The logged row-level propensities take precedence over the nominal 10/80/10 design. If the experiment used quotas, blocks, channel-specific probabilities, or time-varying allocation, those realized design probabilities must be present.

3. Required columns for elasticity-only price decisions

To choose among the tested prices without estimating base conversion, provide the expected contribution obtained **if the customer buys** under every tested price:

Column	Type	Meaning
contribution_H	float	Expected contribution per purchased policy at the high price.
contribution_C	float	Expected contribution per purchased policy at control.
contribution_L	float	Expected contribution per purchased policy at the low price.

Contribution should use the business definition that will govern deployment: displayed premium minus expected claims, commission, taxes, acquisition expense, payment costs, and other arm-dependent variable costs. If only revenue is available, use price revenue but label the resulting policy as a revenue policy rather than a contribution policy.

For customer $\mathbf{X}$, the arm-specific expected contribution is

$$q_t(\mathbf{x}) * \text{contribution}_t(\mathbf{x}).$$

Because $q_t(\mathbf{x}) = q_C(\mathbf{x}) * RR_{tC}(\mathbf{x})$, the common positive factor $q_C(\mathbf{x})$ cancels from the choice among arms:

$$\text{argmax}_t \text{contribution}_t(\mathbf{x}) * RR_{tC}(\mathbf{x}).$$

4. Strongly recommended experiment fields

Column	Purpose
randomization_block	Reproduce blocked or quota-based assignment and randomization tests.
assigned_multiplier	Intended experimental multiplier.
base_premium	Pre-randomization premium or technical tariff.
displayed_premium	Actual premium shown; audit rounding and implementation compliance only. Never use it as an elasticity-model feature.
tariff_version	Detect tariff drift and implementation changes.
channel	Channel/aggregator context, if known before assignment.
purchase_timestamp	Verify the outcome window.
cancelled_early	Optional sensitivity outcome, defined in advance.
expected_claim_cost	Build arm-specific contribution and explain economically meaningful heterogeneity.
tariff_uncertainty	Pre-treatment uncertainty or out-of-distribution score from the tariff model.
competitor_gap	Pre-treatment competitor price gap or rank, if legally and operationally available before assignment.

5. Feature manifest

Supply an explicit feature manifest rather than selecting all columns automatically. At minimum it should contain:

Field	Meaning
feature_name	Input column.
type	numeric, categorical, ordinal, or date-derived.
available_before_assignment	Must be true.
business_definition	Human-readable definition.
missingness_reason	Structural, operational, unknown, or not applicable.
expected_stability	Stable, tariff-version dependent, seasonal, or transient.
approved_for_pricing	Governance decision.

6. Forbidden feature classes

Do not use any variable that reveals or is caused by randomized price assignment or purchase. Examples include:

- assigned arm or multiplier;
- displayed experimental premium;
- price shown after the multiplier;
- purchase, payment method chosen after purchase, policy identifier, or cancellation observed after purchase;
- agent actions made after seeing the experimental price;
- features recomputed after assignment using the displayed premium;
- future claims or renewals.

The treatment arm is the **target** in buyer-flip models and enters the likelihood through its logged propensity. It is never a predictor.

7. Timing and repeated quotes

Document:

1. eligibility timestamp;
2. assignment timestamp;
3. price display timestamp;
4. purchase attribution window;
5. rules for revisited or revised quotes;
6. whether one customer could see different arms;
7. whether manual overrides occurred after randomization.

All rows from one customer or household must remain in the same split. For deployment across time, reserve the most recent eligible period as a final chronological holdout.

8. No synthetic fallback

If the required randomized table is absent, the agent must stop after producing a missing-data report. It may run deterministic unit tests of formulas and software APIs, but it must not generate a synthetic insurance dataset, rank models on generated outcomes, or describe those rankings as evidence about the portfolio.

Part IV — Research log, qualifications, and dead ends

Date: 2026-08-05

Scope and evidence boundary

This continuation deliberately separates three kinds of evidence:

1. **Algebraic identification results**, which follow from Bayes' rule and the randomized assignment mechanism.
2. **Software/API tests**, which use tiny deterministic or generated arrays only to verify formulas, gradients, offsets, cross-validation plumbing, and library compatibility.
3. **Empirical model comparisons**, which require the insurer's real randomized quote-level data and were not performed here because no such table was supplied.

No synthetic model leaderboard is included. No claim is made that LightGBM, XGBoost, Random Forest, or any other flexible model wins on the insurer's portfolio.

Positive findings

Base conversion cancels from the quote-level price choice

Let $g_t(x)$ be contribution if the customer buys at tested arm t , and let $RR_{tC}(x) = q_t(x)/q_C(x)$. Then

$$g_t q_t = q_C * g_t * RR_{tC}.$$

Since $q_C(x) > 0$ is common across arms, the best tested price for that quote is determined by $\arg\max_t g_t RR_{tC}$. This is the central business result: a conversion-scale model is not required to choose among the tested prices, provided arm-specific contribution is known.

Buyer-side Bayes flip exactly identifies relative demand

The buyer posterior satisfies

$$r_t = \pi_t q_t / \sum_j \pi_j q_j,$$

so

$$q_t/q_C = (r_t/\pi_t)/(r_C/\pi_C).$$

This is exact and does not depend on the conversion rate being small. The low conversion rate matters for practical signal-to-noise considerations, not for identification.

Pairwise offset likelihood is a practical main model

For H versus C buyers,

$$\text{logit } P(H \mid H/C, \text{buyer}, X) = \log(\pi_H/\pi_C) + \log RR_{HC}(X).$$

The corresponding identity holds for L versus C. This lets LightGBM and XGBoost use their ordinary binary log-loss objective with a known row-specific initial margin. It avoids a custom multiclass objective and estimates the two risk-ratio contrasts directly.

Inverse-propensity weighting makes standard scikit-learn models usable

Among pairwise buyers, weighting each row by $1/\pi_T$ changes the weighted conditional target to $q_t/(q_t + q_C)$. Its logit is the desired log risk ratio. This supports penalized logistic regression and Random Forest without pretending that class balancing creates information.

Strong shrinkage is structurally necessary

With a 10/80/10 allocation and a roughly 5% conversion rate, the side-arm buyer counts can be small even in a large quote table. Therefore the default hierarchy should start with a global scalar elasticity, then add controlled heterogeneity only when out-of-fold conditional likelihood and forward-time evidence justify it.

Proper likelihood and randomized policy value answer different questions

Buyer conditional NLL evaluates whether risk ratios are numerically correct. Held-out IPS/SNIPS contribution evaluates whether those ratios produce a useful pricing rule. Neither replaces the other. A flexible model should pass both likelihood/calibration gates and a business-value gate.

Important qualifications

Buyer likelihood weights the buyer distribution

The pointwise risk ratio remains identified, but finite-capacity training and model selection emphasize customer regions that generate buyers. This is usually aligned with contribution value. A quote-population-weighted elasticity surface would need covariate-shift weighting, which effectively reintroduces a conversion/density-ratio model and can have high variance.

Elasticity alone is not the same as optimal pricing without margins

The relative demand model identifies where customers are more or less price sensitive. Choosing whether a 10% increase or decrease is financially attractive also requires arm-specific contribution if a purchase occurs. Base conversion cancels, but margin does not.

Three price points identify finite differences, not a global demand curve

The scalar model assumes local constant elasticity over the tested interval. The two-gap model allows asymmetric finite differences. Neither justifies extrapolation outside the tested range without additional experiments or structural assumptions.

Marginal asymmetry is not proof of individual curvature

Different H/C and L/C effects can arise from mixtures of customers with heterogeneous constant elasticities. The two-gap model is operationally useful, but its difference should not automatically be interpreted as each individual's nonlinear demand curve.

Random Forest is not automatically more robust

A forest can model nonlinear heterogeneity, but its leaf probabilities are unstable when side-arm buyer counts are small. Large leaves, limited depth, inverse-propensity weighting, and out-of-fold affine calibration are essential. Impurity importance is not an acceptable basis for business interpretation.

Dead ends and rejected shortcuts

- Synthetic model tournaments for selecting the production architecture.
- Individual elasticity RMSE or correlation on real three-arm data; individual truth is unobserved.
- AUC, VUS, accuracy, or F1 as primary tuning criteria; they do not identify calibrated probability ratios.
- `class_weight='balanced'`, SMOTE, or duplicated buyers without exact posterior correction.
- One-versus-rest probabilities treated as a coherent three-arm posterior.
- Using displayed experimental premium or assigned multiplier as a predictor when arm is the target.
- Random row CV when customers or households repeat.
- Fitting calibration on training predictions.
- Optimizing high-variance policy value directly inside every Optuna trial.
- Large unrestricted two-gap trees before the global scalar model is beaten.
- Selecting a model because it has attractive in-sample segment plots.

Implementation discoveries and corrections

- Pairwise XGBoost and LightGBM wrappers include the global log-risk-ratio intercept as well as the row-specific propensity offset. Unit tests confirm that a no-split model recovers the global conditional-likelihood intercept.

- The Optuna CV helper originally passed early-stopping keyword arguments to every estimator. A new signature-aware fitting helper now passes only supported arguments, so scikit-learn/Random-Forest and native boosting models use the same objective safely.
- The scikit-learn 1.8 logistic API deprecates the older `penalty` selector. The search now tunes `l1_ratio` directly: 0 gives L2, 1 gives L1, and intermediate values give elastic net.
- Optuna's multivariate/group TPE options are still documented as experimental in the current stable documentation. The configuration is retained because it handles conditional spaces well, but versions and warnings must be recorded.
- XGBoost and LightGBM custom scalar objectives use a positive Fisher/Gauss-Newton Hessian because the exact raw-score Hessian can become negative after the softplus positivity transform.

Software verification completed

- Python compilation passed.
- 22 unit and smoke tests passed.
- Statement coverage was 77% across the package during the final test run.
- Native LightGBM and XGBoost pairwise and scalar wrappers were smoke-tested against the locally installed APIs.
- The four example scripts expose working CLI interfaces.
- The audit, one-trial pairwise tuning, one-trial scalar tuning, and OOF evaluation scripts were executed end-to-end on a temporary API fixture. The fixture and its numerical outputs are not included and are not evidence about insurance model quality.

Unresolved empirical questions for the real-data agent

1. Does any heterogeneous model improve conditional NLL over the global scalar/two-gap anchor with a customer-clustered confidence interval?
2. Is the improvement stable in a strictly later tariff period?
3. Does the H/C contrast contain enough high-price buyers to support nonlinear segmentation?
4. How much monotonicity projection is required before and after calibration?
5. Are the top price-increase and price-decrease sets stable across folds, seeds, tariff versions, and channels?
6. Does the model policy improve held-out contribution relative to control and the best uniform tested multiplier under IPS/SNIPS?
7. Do competitor-position and tariff-uncertainty variables explain stable elasticity variation, or only transient operational artifacts?
8. Is one scalar elasticity sufficient, or is two-gap asymmetry reproducible and decision-relevant?

These questions cannot be resolved without the actual randomized quote-level table.

Part V — Software QA and evidence boundary

Scope

This release is a reference implementation and an agent playbook for a real randomized motor-insurance price experiment. It contains no generated insurance dataset and no synthetic model-performance leaderboard. Deterministic arrays and a temporary API fixture were used only to test identities, derivatives, library interfaces, and command-line plumbing.

Environment

See `ENVIRONMENT.json` for the exact locally tested versions. The release was tested with Python 3.13, scikit-learn 1.8.0, LightGBM 4.6.0, XGBoost 3.1.3, and Optuna 4.8.0.

Automated checks

- Python bytecode compilation completed for `elasticity_only/`, `examples/`, and `tests/`.
- `pytest -q` completed with **22 passed** and no failed tests.
- The only warnings were Optuna warnings that `TPESampler(multivariate=True, group=True)` remains an experimental interface.
- Statement coverage was **77%** over the package and test suite. See `COVERAGE.txt`.
- Native LightGBM and XGBoost pairwise-offset and scalar-custom-objective wrappers were included in smoke tests.
- Tests verify the Bayes identity, risk-ratio reconstruction, business decision cancellation, monotonic projections, offset recovery, custom gradients, policy estimators, split integrity, preprocessing, tuning helpers, calibration, multiclass reconstruction, stacking, and schema validation.

End-to-end interface checks

The four example programs were executed against a temporary, deterministic API fixture created solely for software validation:

1. experiment audit;
2. one-trial pairwise Optuna tuning;
3. one-trial scalar Optuna tuning;
4. evaluation of out-of-fold prediction-file structure and metrics.

The fixture and its numerical model outputs are intentionally not included. They are not evidence about motor-insurance demand and must not be used to rank model families.

Evidence boundary

What is verified:

- mathematical identities implemented by the code;
- gradient and Hessian consistency at tested points;
- compatibility with the locally installed library APIs;
- output-shape, validation, persistence, and split invariants.

What is not verified without the user's data:

- existence or strength of heterogeneous elasticity;
- which model family wins;
- business lift of any personalized pricing policy;
- temporal stability across tariff versions;
- adequacy of the supplied feature set;
- causal validity of an experiment whose operational logs have not been audited.

Production-hardening limitations

This is a high-quality research reference, not a fully managed production service. Before deployment, add organization-specific security review, data-access controls, model registry integration, monitoring, protected-feature governance, deterministic dependency locking, distributed-training tests, and larger failure-path coverage. Run all model training inside the organization's approved environment and retain the exact input hash, feature manifest, folds, Optuna storage, model binaries, and out-of-fold predictions.

References

1. Dai, J. Y., Liang, C. J., LeBlanc, M., & Kooperberg, C. (2018). **Case-only approach to identifying markers predicting treatment effects on the relative risk scale in randomized trials.** *Biometrics*, 74(2), 753–763. DOI: 10.1111/biom.12789. <https://pubmed.ncbi.nlm.nih.gov/28960244/>
2. Xu, Y., & Yadlowsky, S. (2022). **Calibration Error for Heterogeneous Treatment Effects.** *Proceedings of AISTATS 2022*, PMLR 151:9280–9303. <https://proceedings.mlr.press/v151/xu22c.html>
3. Yadlowsky, S., Fleming, S., Shah, N., Brunskill, E., & Wager, S. (2025). **Evaluating Treatment Prioritization Rules via Rank-Weighted Average Treatment Effects.** *Journal of the American Statistical Association*, 120(549), 38–51. DOI: 10.1080/01621459.2024.2393466. <https://pubmed.ncbi.nlm.nih.gov/40248684/>
4. Dudík, M., Langford, J., & Li, L. (2011). **Doubly Robust Policy Evaluation and Learning.** ICML. <https://arxiv.org/abs/1103.4601>
5. scikit-learn developers. **StratifiedGroupKFold.** https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html
6. scikit-learn developers. **Probability calibration user guide.** <https://scikit-learn.org/stable/modules/calibration.html>
7. scikit-learn developers. **CalibratedClassifierCV.** <https://scikit-learn.org/stable/modules/generated/sklearn.calibration.CalibratedClassifierCV.html>
8. scikit-learn developers. **RandomForestClassifier.** <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
9. scikit-learn developers. **Permutation feature importance.** https://scikit-learn.org/stable/modules/permutation_importance.html
10. LightGBM developers. **Parameters Tuning.** <https://lightgbm.readthedocs.io/en/stable/Parameters-Tuning.html>
11. LightGBM developers. **Dataset API: init_score and categorical features.** <https://lightgbm.readthedocs.io/en/stable/pythonapi/lightgbm.Dataset.html>
12. XGBoost developers. **Advanced Usage of Custom Objectives.** https://xgboost.readthedocs.io/en/stable/tutorials/advanced_custom_obj.html
13. XGBoost developers. **Custom Objective and Evaluation Metric.** https://xgboost.readthedocs.io/en/stable/tutorials/custom_metric_obj.html
14. XGBoost developers. **XGBoost Parameters.** <https://xgboost.readthedocs.io/en/stable/parameter.html>
15. Optuna developers. **TPESampler.** <https://optuna.readthedocs.io/en/stable/reference/samplers/generated/optuna.samplers.TPESampler.html>
16. Optuna developers. **MedianPruner.** <https://optuna.readthedocs.io/en/stable/reference/generated/optuna.pruners.MedianPruner.html>